

DSA Sorting Intuition and Task 2 Guide

How array shape affects efficiency, and how to improve quicksort

This document is designed to build intuition from the actual sorting algorithms in the coursework repository. It explains what each algorithm is naturally good at, what kinds of arrays make it struggle, why small arrays require special thinking, and how to complete Task 2 systematically. ¹

The central idea is:

An algorithm is not simply “fast” or “slow.” Its work depends on the structure of the input and on the decisions made during the algorithm.

1. What is an array’s “shape”?

An array’s shape means the arrangement of its values, not its physical size. Two arrays can contain the same values and have the same length but cause very different behaviour in a sorting algorithm.

Array shape	Example	Description
Already sorted	[1, 2, 3, 4, 5]	Every value is already in ascending order.
Reverse sorted	[5, 4, 3, 2, 1]	Every value is in descending order.
Random	[3, 1, 5, 2, 4]	Values have no useful order.
Nearly sorted	[1, 2, 4, 3, 5]	Only a few values are misplaced.
95%-sorted	Most values follow ascending order, but a percentage is randomly chosen.	The amount of disorder depends on how the benchmark constructs the data.
Many duplicates	[4, 2, 4, 1, 4]	Several values are equal.
Small array	[5, 2, 4]	The fixed overhead of an algorithm becomes important.

The repository’s benchmark generates three main conditions: random input, 95%-sorted input, and already sorted input. 1 A useful defense answer should always specify **which input condition** is being discussed.

2. The three questions to ask about any algorithm

When trying to understand an algorithm, ask these questions:

Question 1: What does the algorithm look at?

Insertion sort looks backward through a sorted prefix. Merge sort repeatedly looks at the front of two sorted halves. Quicksort looks at values in relation to a pivot.

Question 2: What does the algorithm move or repeat?

Insertion sort shifts larger values. Merge sort copies and merges values at every recursion level. Quicksort repeats partitioning on smaller subarrays.

Question 3: What input arrangement causes the most work?

The answer depends on the algorithm. An arrangement that is excellent for insertion sort can be dangerous for basic quicksort.

Algorithm	Main source of work	Input arrangement that commonly helps	Input arrangement commonly hurts
Insertion sort	Shifting values left or right	Already sorted or nearly sorted	Random or reverse sorted
Merge sort	Splitting, copying, and merging	All arrangements have similar growth	No dramatic differences in this implementation
Basic quicksort	Partition balance and recursion	Inputs producing balanced pivots	Sorted input or element i
Improved quicksort	Pivot quality plus recursion overhead	Randomised or balanced inputs	Adversarial inputs can still be bad

3. Insertion sort intuition

3.1 How it works

Insertion sort maintains a sorted prefix:

Plain Text

[sorted prefix | unsorted remainder]

For this array:

Plain Text

[5, 2, 4, 1, 3]

The first value is treated as sorted:

Plain Text

[5 | 2, 4, 1, 3]

It picks 2, shifts 5 right, and inserts 2:

Plain Text

[2, 5 | 4, 1, 3]

It then picks 4, shifts 5, and inserts 4:

Plain Text

[2, 4, 5 | 1, 3]

The sorted prefix grows after every outer-loop iteration.

3.2 Best array for insertion sort

The best arrangement is already sorted:

Plain Text

```
[1, 2, 3, 4, 5]
```

For each new value, insertion sort checks the value to its left, finds that it is not greater, and performs no shifts. It still performs comparisons, so the time is not zero. The work grows approximately linearly:

Plain Text

$O(n)$

A nearly sorted array is also good when only a small amount of movement is required:

Plain Text

```
[1, 2, 3, 5, 4, 6, 7]
```

Only 5 must move past 4.

3.3 Difficult arrays for insertion sort

A reverse-sorted array is difficult:

Plain Text

```
[5, 4, 3, 2, 1]
```

The value 4 moves past 5. Then 3 moves past 5 and 4. Then 2 moves past three values. The number of movements is approximately:

Plain Text

$1 + 2 + 3 + \dots + (n - 1)$

This sum grows like n^2 , giving:

Plain Text

$O(n^2)$

Random arrays usually also require many shifts, so their observed behaviour is generally approximately quadratic.

3.4 A better mental measure: inversions

An inversion is a pair of values that are in the wrong order. In:

Plain Text

[3, 1, 2]

there are two inversions: (3,1) and (3,2) .

Insertion sort performs work related to the number of inversions. Few inversions mean few shifts. Many inversions mean many shifts. This explains why “nearly sorted” input can be much faster than random input even when both have the same length.

4. Merge sort intuition

4.1 How it works

Merge sort does not insert values one at a time and does not choose a pivot. It uses divide and conquer:

Plain Text

split → split → split → merge → merge → merge

For:

Plain Text

[8, 3, 5, 4, 7, 6, 1, 2]

it eventually creates one-element sections. One-element sections are already sorted. It then merges sorted sections:

Plain Text

[8] + [3] → [3, 8]

[5] + [4] → [4, 5]

The merge operation always compares the front values of two sorted sections and copies the smaller one into the result.

4.2 Which arrays are good for merge sort?

Merge sort has no major input arrangement that makes its asymptotic growth collapse. For random, nearly sorted, reverse-sorted, and already sorted arrays, it still divides into halves and performs merging work. Therefore, its growth is approximately:

Plain Text

$O(n \log n)$

Already sorted input may have a small practical advantage because comparisons may be predictable, but the unoptimised `Merge.java` still performs its normal copying and merging process. Do not confuse a small timing difference with a different complexity.

4.3 Why merge sort is stable in this implementation

When the two current values are equal, the code chooses the value from the left half:

Java

```
else a[k] = aux[i++];
```

Choosing the earlier equal value preserves the original relative order of equal elements. This property is called stability. The coursework sorts integers, so stability may not be visible, but it is still a useful implementation detail.

5. Basic quicksort intuition

5.1 How it works

Basic quicksort chooses a pivot and partitions the current subarray around it. If the pivot is 6 :

Plain Text

[2, 8, 3, 6, 1, 7, 5]

a successful partition places smaller values to the left and larger values to the right:

Plain Text

```
[2, 3, 1, 5] [6] [8, 7]
```

The two sides are then sorted recursively.

5.2 The ideal pivot

The ideal pivot divides the current subarray into two roughly equal parts. For an eight-element subarray, a pivot near the middle produces sections of about three or four elements:

Plain Text

```
left:  3 or 4 values
pivot: 1 value
right: 3 or 4 values
```

Each recursive level processes approximately n values, and there are approximately $\log n$ levels, giving average behaviour near:

Plain Text

```
 $O(n \log n)$ 
```

5.3 The dangerous pivot

The dangerous pivot is repeatedly the smallest or largest value. In the repository, the original pivot is the first element. On sorted input:

Plain Text

```
[1, 2, 3, 4, 5, 6, 7]
```

1 is chosen first, producing:

Plain Text

```
[] [1] [2, 3, 4, 5, 6, 7]
```

Then 2 is chosen from the remaining section, and so on. The recursion becomes a long chain rather than a balanced tree:

Plain Text

7 → 6 → 5 → 4 → 3 → 2 → 1

This produces approximately $O(n^2)$ work.

5.4 Why input shape matters more for quicksort

Quicksort's performance depends heavily on partition balance. The input arrangement matters because it affects the pivot's position relative to the other values. Merge sort's division is controlled by indexes, while quicksort's division depends on values and pivot selection.

6. Why small arrays are special

Complexity describes growth for large n , but real performance also includes constant overhead. A recursive quicksort call may need to:

1. Create a method call.
2. Calculate boundaries.
3. Partition the subarray.
4. Scan values.
5. Make more recursive calls.

For a subarray of only three or five values, this administrative work can be large compared with the sorting work itself.

Insertion sort has a simple loop and little setup. Therefore, even though insertion sort has quadratic worst-case complexity, it can be faster than quicksort on **small** subarrays.

This is not a contradiction. Complexity and practical speed are different ideas:

Situation	Best question
Very large input	How does the work grow as n grows?
Very small input	Which implementation has less overhead?
Coursework benchmark	Which configuration is faster on the measured sizes and still correct?

The insertion cutoff uses this idea:

Plain Text

large subarray → quicksort
small subarray → insertion sort

7. Task 2: the problem to solve

The original `Quick.java` has three unfinished improvements marked by `TODO` comments. 2
The starting configuration in `Bench.java` is:

Java

```
new Quick(false, false, 0)
```

This means:

Setting	Meaning
false	Do not shuffle first.
false	Do not use median-of-three.
0	Do not use an insertion-sort cutoff.

Task 2 asks you to implement the improvements one at a time, test them, test combinations, determine the best cutoff, and record the results in `answers.txt`. 1

The goal is not simply to make code look more sophisticated. The goal is to establish, with measurements, whether each decision improves performance and how it changes behaviour.

8. Improvement 1: shuffle the array once

8.1 Why shuffle?

The original quicksort always takes the first element as pivot. If the input is sorted, the first element is the smallest. If the input is reverse sorted, the first element is the largest. Both cases create highly unbalanced partitions.

Shuffling changes this:

Plain Text

Before: [1, 2, 3, 4, 5, 6, 7]

After: [4, 1, 6, 2, 7, 3, 5]

Now the first pivot is likely to be somewhere in the middle rather than at an extreme.

8.2 How to implement it

The repository already provides:

Java

```
private static void shuffle(int[] a)
```

In the public `sort` method, use it only when the option is enabled:

Java

```
public void sort(int[] a) {  
    if (shuffleFirst) {  
        shuffle(a);  
    }  
  
    sort(a, 0, a.length - 1);  
}
```

The shuffle must occur **once**, before the recursive sorting begins. Do not place it inside the recursive `sort(a, lo, hi)` method. Shuffling every recursive subarray would add unnecessary work and would not follow the task instruction.

8.3 What shuffling changes and does not change

Shuffling changes the input order and reduces the probability of a bad pivot sequence. It does not guarantee that every pivot is perfect. It usually improves expected behaviour on hostile or predictable inputs, moving the observed pattern toward average $O(n \log n)$.

9. Improvement 2: median-of-three pivot selection

9.1 Why use three candidates?

Instead of trusting the first value, examine:

Plain Text

first, middle, last

For:

Plain Text

[1, 3, 4, 6, 8, 10, 12]

the candidates are 1, 6, and 12. Their median is 6, which is much more central than the first value 1.

For a reverse-sorted subarray:

Plain Text

[12, 10, 8, 6, 4, 3, 1]

the candidates are 12, 6, and 1; the median is again 6.

9.2 How to implement it in this repository

The existing helper is:

Java

```
int medianOfThree(int[] a, int i, int j, int k)
```

It returns the index containing the median value. The partition method expects the pivot to be in `a[lo]`, so the implementation should conceptually do:

Java

```
int mid = lo + (hi - lo) / 2;  
int medianIndex = medianOfThree(a, lo, mid, hi);  
exchange(a, lo, medianIndex);
```

After the exchange, the rest of the partition code can continue using:

Java

```
int pivot = a[lo];
```

The important detail is that `mid` must be calculated for the **current subarray**, not for the whole original array.

9.3 What median-of-three changes

It improves the pivot-selection decision for each recursive subarray. It is especially helpful when the first, middle, and last positions reveal that the first value is an extreme. It does not guarantee a perfectly balanced partition because three samples cannot describe every value in the subarray.

9.4 Median-of-three versus shuffling

These improvements attack similar symptoms in different ways:

Feature	Shuffle first	Median-of-three
When it acts	Once, before sorting	During partitioning of each subarray
What it changes	The order of all values	The pivot candidate selection
Main strength	Protects against predictable global input order	Avoids obvious poor pivots using local information
Main cost	Shuffles the entire array	Examines three values and may exchange one
Guarantee	Randomisation reduces probability of bad sequence	Three samples do not guarantee balance

They can be used separately or together. The benchmark must tell you whether the combination is worthwhile on your machine.

10. Improvement 3: insertion sort cutoff

10.1 Why switch algorithms?

Quicksort is efficient for large sections, but recursive partitioning has overhead. Insertion sort is simple and often efficient for a small section, especially when that section is already partly ordered as a result of previous partitions.

The improved recursive method follows this decision:

Plain Text

```
if the current subarray is small:
    insertion sort it
else:
    partition it and recurse
```

10.2 How to implement it

The current subarray is `a[lo..hi]`, so its size is:

Java

```
hi - lo + 1
```

A logically clear version is:

Java

```
if (hi - lo + 1 <= insertionSortCutoff) {
    Insertion.sort(a, lo, hi);
    return;
}
```

Your exact comparison should be consistent with the interpretation of “below the cutoff” in the lab instructions. The essential requirements are that the size is calculated correctly, insertion sort is called on the current range, and the method returns immediately afterward.

10.3 Why the `return` matters

Suppose insertion sort has already sorted `a[lo..hi]`. If the method does not return, quicksort will partition and recursively process that same range again. That wastes work and can cause incorrect control flow.

10.4 Choosing the cutoff scientifically

Do not guess a cutoff and call it best. Test a range:

Plain Text

0, 5, 10, 20, 30, 40, 50, 60

Keep the other settings fixed while testing cutoff values. For example, compare:

Java

```
new Quick(true, true, 0)
new Quick(true, true, 10)
new Quick(true, true, 20)
new Quick(true, true, 30)
new Quick(true, true, 40)
```

Choose the value with the best overall performance. Then test nearby values above and below it. A convincing conclusion is stronger if the performance becomes worse when the cutoff is moved in either direction.

10.5 Why too small or too large can be bad

If the cutoff is 0 or 1, quicksort continues recursion almost to the bottom, so it receives little benefit from insertion sort.

If the cutoff is extremely large, insertion sort handles sections that are no longer small. Since insertion sort can be quadratic, excessive use of it can slow the program.

The best cutoff balances:

Plain Text

```
quicksort's partitioning and recursion overhead
against
insertion sort's simple but quadratic local work
```

11. How to pair and compare versions fairly

A fair comparison changes one meaningful factor at a time.

11.1 Baseline first

Keep the original line:

Java

```
executionTimeReport("Quick.java: quicksort", new Quick(false, false, 0)::sort);
```

This is the baseline. Every improvement should be compared against it.

11.2 One-improvement comparisons

Run these versions:

Version	Constructor
Baseline	<code>new Quick(false, false, 0)</code>
Shuffle only	<code>new Quick(true, false, 0)</code>
Median-of-three only	<code>new Quick(false, true, 0)</code>
Cutoff only	<code>new Quick(false, false, 20)</code>

These comparisons answer: **What does each improvement do by itself?**

11.3 Pairwise combinations

Then compare pairs:

Version	Constructor
Shuffle + median-of-three	<code>new Quick(true, true, 0)</code>
Shuffle + cutoff	<code>new Quick(true, false, 20)</code>
Median-of-three + cutoff	<code>new Quick(false, true, 20)</code>

These comparisons answer: **Do two improvements cooperate, or does one make the other unnecessary?**

11.4 All three

Finally test:

Java

```
new Quick(true, true, chosenCutoff)
```

Do not assume this must be fastest. Each improvement has a cost. Shuffling costs time, median-of-three performs extra comparisons, and insertion cutoff may be too high or too low. The best configuration is the one supported by the benchmark while still passing correctness tests.

12. Use valid array examples to understand each improvement

Example A: sorted input

Plain Text

```
[1, 2, 3, 4, 5, 6, 7]
```

Basic first-pivot quicksort:

Plain Text

```
pivot 1 → empty left, six-value right  
pivot 2 → empty left, five-value right
```

Shuffling might produce:

Plain Text

```
[4, 1, 6, 2, 7, 3, 5]
```

Median-of-three on the original sorted range chooses 4 from candidates 1, 4, and 7. Both strategies try to prevent the chain of empty partitions.

Example B: reverse-sorted input

Plain Text

```
[7, 6, 5, 4, 3, 2, 1]
```


The first pivot 7 is the largest, so the right partition is empty. Median-of-three examines 7, 4, and 1, chooses 4, and creates a more balanced division.

Example C: nearly sorted input

Plain Text

```
[1, 2, 3, 5, 4, 6, 7]
```

Insertion sort handles this very well because only 5 and 4 need to be corrected. Basic quicksort may also perform adequately, but its result depends on the pivot choices.

Example D: a small partition

Plain Text

```
[2, 5, 1, 4, 3]
```

If this is a small quicksort subarray and the cutoff is at least 5, use insertion sort directly. There is no benefit in repeatedly partitioning such a small section unless benchmarking proves otherwise.

Example E: duplicate values

Plain Text

```
[4, 2, 4, 1, 4, 3]
```

Correctness must include duplicates. The final result should be:

Plain Text

```
[1, 2, 3, 4, 4, 4]
```

A partition implementation must not lose, duplicate, or incorrectly order equal values.

13. How to verify correctness before efficiency

A timing improvement is meaningless if the output is wrong. `Bench.java` tests arrays of many sizes and compares the result with Java's `Arrays.sort` output. ³

Test at least these categories mentally or with small custom arrays:

Test	Example
Empty	<code>[]</code>
One value	<code>[4]</code>
Two values sorted	<code>[2, 5]</code>
Two values reversed	<code>[5, 2]</code>
Already sorted	<code>[1, 2, 3, 4]</code>
Reverse sorted	<code>[4, 3, 2, 1]</code>
Duplicates	<code>[3, 1, 3, 2, 1]</code>
Negative values	<code>[-2, 4, -1, 0]</code>
Random	<code>[5, 1, 4, 2, 3]</code>

If the program reports a failure, use the printed input, expected result, and actual result. Reproduce the small case in a temporary `main` method and trace the indexes. Remove temporary debugging code before submission.

14. How to interpret the benchmark output

For each configuration, compare:

1. Random input.
2. 95%-sorted input.
3. Sorted input.
4. Small and large input sizes.
5. Total time and per-item time.

A useful results table is:

Configuration	Random, large <code>n</code>	95%-sorted, large <code>n</code>	Sorted, large <code>n</code>	Correct?
---------------	------------------------------	----------------------------------	------------------------------	----------

Baseline	record	record	record	yes/no
Shuffle only	record	record	record	yes/no
Median-of-three only	record	record	record	yes/no
Cutoff only	record	record	record	yes/no
All three	record	record	record	yes/no

Do not compare one configuration using `n = 1,000` against another using `n = 100,000`. Keep the input sizes and input conditions the same.

When timing results are close, run the comparison again. Look for consistent patterns rather than treating one tiny difference as proof.

15. A complete Task 2 workflow

Step 1: Record the baseline

Compile and run the original program. Save its output. Confirm that the original algorithms sort correctly.

Step 2: Implement shuffling only

Complete the `shuffleFirst` branch. Test `new Quick(true, false, 0)`. Compare sorted-input performance with the baseline.

Step 3: Implement median-of-three only

Complete the median selection and exchange. Test `new Quick(false, true, 0)`. Check sorted and reverse-sorted arrays carefully.

Step 4: Implement insertion cutoff

Complete the small-subarray condition and call `Insertion.sort(a, lo, hi)`. Test several cutoff values, initially with the other improvements disabled.

Step 5: Test combinations

Compare pairs and then all three improvements. Keep labels clear in `Bench.java` so the output can be interpreted.

Step 6: Select and validate the cutoff

Use a systematic range of values. Test values on both sides of the selected value and repeat close comparisons.

Step 7: Choose the best configuration

Choose the configuration with the best reliable performance across relevant input types, provided it passes all correctness tests.

Step 8: Complete `answers.txt`

Explain which improvements affected complexity, what cutoff worked best, how the cutoff was found, and which combination performed best. Use measured evidence rather than unsupported claims.

Step 9: Produce final output

Run the final benchmark and save it as `output.txt`. Keep the baseline benchmark line as requested by the lab instructions. 1

16. The deepest intuition: balance versus overhead

The entire lab can be understood through two competing ideas.

Quicksort's balance problem

Quicksort wants balanced partitions. Shuffling and median-of-three try to prevent the recursion tree from becoming a long chain.

Plain Text

```
balanced:      n → n/2 + n/2 → n/4 sections
unbalanced:    n → n-1 → n-2 → n-3
```

Balanced recursion tends toward $O(n \log n)$. Repeatedly unbalanced recursion tends toward $O(n^2)$.

Small-section overhead problem

Even with good partitions, eventually quicksort creates small sections. At that point, recursive calls and partition scanning can cost more than the useful sorting work. The insertion cutoff removes that overhead.

Therefore:

Plain Text

```
shuffle / median-of-three = improve partition balance
insertion cutoff = reduce small-subarray overhead
```

These are different improvements solving different problems.

17. How to explain the final result in a defense

A strong explanation is:

“The input arrangement affects each algorithm differently. Insertion sort benefits from sorted or nearly sorted arrays because it performs few shifts, but random input creates many shifts and approximately quadratic behaviour. Merge sort has approximately $O(n \log n)$ growth across the input conditions because it divides by index and merges every level. Basic quicksort is sensitive to pivot balance; choosing the first element makes already sorted or reverse-sorted input dangerous. In Task 2, shuffling reduces predictable bad orders, median-of-three selects a more central pivot from local candidates, and the insertion cutoff avoids recursive overhead on small subarrays. I compared the baseline, each improvement individually, pairs, and the full combination using the same input conditions and checked correctness before selecting the best configuration.”

The most important mindset is not to ask only, “Which algorithm is best?” Ask:

Plain Text

```
best for which input shape?
best at what input size?
```

best under which configuration?
what evidence supports the conclusion?

That is the intuition the lab is designed to develop.

References

- [1] Repository README: Lab 1: Sorting, Complexity
- [2] Repository Quick.java
- [3] Repository Bench.java